AGENTS.md - Grafana Dashboard Development

Overview

This directory contains Grafana dashboards managed as code using Grafonnet (based on Jsonnet). Dashboards are automatically uploaded to https://dashboards.gitlab.net on master builds.

Creating a New Dashboard

Step 1: Create the Grafana Folder (if needed)

If the folder doesn't exist yet:

```
cd dashboards
./create-grafana-folder.sh <folder_uid> '<Folder Title>'
# Example: ./create-grafana-folder.sh my-service 'My Service'
```

Requires `GRAFANA_API_TOKEN` environment variable.

Step 2: Create the Dashboard File

Create a new file following the naming convention:

```
dashboards/<folder_name>/<dashboard_name>.dashboard.jsonnet
```

Step 3: Basic Dashboard Structure

Minimal dashboard using serviceDashboard (recommended for service dashboards):

```
local serviceDashboard = import 'gitlab-dashboards/service_dashboard.libsonnet';

serviceDashboard.overview('<service-type>')
.overviewTrailer()
```

Custom dashboard structure:

```
local grafana = import 'github.com/grafana/grafonnet-lib/grafonnet/grafana.libsonnet';
local basic = import 'grafana/basic.libsonnet';
local layout = import 'grafana/layout.libsonnet';
local row = grafana.row;
local template = grafana.template;
local panel = import 'grafana/time-series/panel.libsonnet';
local target = import 'grafana/time-series/target.libsonnet';

serviceDashboard.overview('<service-type>', startRow=1)
.addTemplate(
  template.custom(
    'environment',
```

```
    'gprd,gstg,ops',
    'gprd',
  ),
)
.addPanels(
  layout.grid([
    basic.statPanel(
      title='',
      panelTitle='My Stat',
      query='my_metric{environment="$environment"}',
      legendFormat='{{ label }}',
    ),
    panel.timeSeries(
      title='My Time Series',
      query='rate(my_counter{environment="$environment"}[$__interval])',
      legendFormat='{{ label }}',
      format='ops'
    ),
  ], cols=2, rowHeight=10, startRow=0),
)
.addPanel(
  row.new(title='Details', collapse=true)
  .addPanels(
    layout.grid([
      // panels here
    ], cols=2, rowHeight=10, startRow=0),
  ),
  gridPos={ x: 0, y: 100, w: 24, h: 1 },
)
.overviewTrailer()
```

Common Imports

```
// Core Grafana
local grafana = import 'github.com/grafana/grafonnet-lib/grafonnet/grafana.libsonnet';
local row = grafana.row;
local template = grafana.template;

// GitLab Dashboard Helpers
local serviceDashboard = import 'gitlab-dashboards/service_dashboard.libsonnet';
local basic = import 'grafana/basic.libsonnet';
local layout = import 'grafana/layout.libsonnet';
local thresholds = import 'gitlab-dashboards/thresholds.libsonnet';

// Time Series Panels
local panel = import 'grafana/time-series/panel.libsonnet';
```

```
local target = import 'grafana/time-series/target.libsonnet';
local threshold = import 'grafana/time-series/threshold.libsonnet';

// Rails/Workhorse specific
local railsCommon = import 'gitlab-dashboards/rails_common_graphs.libsonnet';
local workhorseCommon = import 'gitlab-dashboards/workhorse_common_graphs.libsonnet';
```

Common Panel Types

Stat Panel

```
basic.statPanel(
  title='',
  panelTitle='Panel Title',
  query='my_metric{environment="$environment"}',
  legendFormat='{{ label }}',
  colorMode='value',  // or 'background'
  textMode='value',   // or 'name'
  unit='short',       // 'percentunit', 'ops', 'ms', etc.
  color=[
    { color: 'red', value: 0 },
    { color: 'green', value: 1 },
  ],
)
```

Time Series Panel

```
panel.timeSeries(
  title='Panel Title',
  description='Optional description',
  query='rate(my_metric{environment="$environment"}[$__interval])',
  legendFormat='{{ label }}',
  format='ops'  // 'ms', 'percentunit', 'short', etc.
)
.addTarget(
  target.prometheus(
    'another_metric{environment="$environment"}',
    legendFormat='{{ label }}',
  )
)
.addThreshold(
  threshold.warningLevel(100),
)
```

Layout

Use `layout.grid()` to arrange panels:

```
layout.grid([
  panel1,
  panel2,
  panel3,
], cols=3, rowHeight=10, startRow=0)
```

Template Variables

Standard template variables available:

- `$environment` - gprd, gstg, ops, etc.
- `$stage` - main, canary
- `$PROMETHEUS_DS` - Prometheus data source

Custom template:

```
.addTemplate(
  template.custom(
    'environment',
    'gprd,gstg,ops',
    'gprd',  // default
  ),
)
```

Testing a Dashboard

```
cd dashboards
# Upload to playground (requires GRAFANA_API_TOKEN or 1Password CLI)
./test-dashboard.sh <folder>/<name>.dashboard.jsonnet

# Dry run - output JSON only
cd dashboards
./test-dashboard.sh -D <folder>/<name>.dashboard.jsonnet
```

Note:

- Snapshots can only be created for dashboards that already exist in Grafana. For new dashboards, merge a basic version first
- Snapshots with dynamic variables will not show any data. To fully test the dashboard, a copy in to Playground folder will have to be uploaded. To obtain json for upload, run the script in `Generating Dashboard JSON` section

Generating Dashboard JSON

```
cd dashboards
./generate-dashboard.sh <folder>/<name>.dashboard.jsonnet
# Output goes to dashboards/generated/
```

Never run `./generate-dashboards.sh` script as it will generate all of the defined dashboards!

Shared Dashboards

For multiple dashboards from one file, use `.shared.jsonnet`:

```
{
  "dashboard_uid_1": { /* Dashboard */ },
  "dashboard_uid_2": { /* Dashboard */ },
}
```

Protecting Dashboards

To protect a dashboard from automatic deletion:

1. Add the label `protected` to your dashboard, OR
2. Add to `protected-grafana-dashboards.jsonnet`

Dependencies

Before working on dashboards:

```
# Install asdf plugins (go-jsonnet, jsonnet-bundler)
# See root README.md for setup instructions

# Install vendor dependencies
jb install
```

PromQL Testing

Use Grafana Explore to test queries: https://dashboards.gitlab.net/explore

File Locations

- Dashboard source: `dashboards/<folder>/<name>.dashboard.jsonnet`
- Shared libraries: `../libsonnet/`
- Vendor libraries: `../vendor/`
- Metrics catalog: `../metrics-catalog/`